

OSSP PRACTICAL Q7:-

7. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Make use of the `/proc/maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Code:-

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;
static int static_var = 20;

void code_function()
{
    printf("Code : %p\n", (void *)code_function);
}

int main()
{
    int stack_var = 30;
    static int local_static = 40;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 50;

    printf("Code : %p\n", (void *)code_function);
    printf("Global : %p\n", (void *)&global_var);
    printf("Static : %p\n", (void *)&static_var);
    printf("Static local : %p\n", (void *)&local_static);
    printf("Heap : %p\n", (void *)heap_var);
    printf("Stack : %p\n", (void *)&stack_var);

    printf("\nPID: %d\n", getpid());

    printf("\n/proc/%d/maps:\n", getpid());

    char cmd[100];
    sprintf(cmd, "cat /proc/%d/maps", getpid());
    system(cmd);

    free(heap_var);

    return 0;
}
```

OUTPUT:-

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;
static int static_var = 20;

void code_function()
{
    printf("Code    : %p\n", (void *)code_function);
}

int main()
{
    int stack_var = 30;
    static int local_static = 40;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 50;
```

```
apple@apples-MacBook-Pro-2 practical % ./p7
```

```
Code    : 0x10bb5ada0
Global  : 0x10bb5f000
Static  : 0x10bb5f008
Static local : 0x10bb5f004
Heap    : 0x600000dfc030
Stack   : 0x7ff7b43a8778
```

```
PID: 1063
```

```
/proc/1063/maps:
```